

Úvod do jazyka Python

Třídy a strutury

Vladimír Župka 2021

Použité zdroje informací	4
Třída (class)	5
Ukázka jednoduché třídy	5
Instance třídy	5
Terminologie tříd	6
Úprava třídy	6
Úprava instance	6
Statické a třídní metody	7
Televizor a ovladač	8
Test televizoru a ovladače	8
Vstupní parametry třídy a instanční proměnné	9
Datové struktury	10
Jednostranně propojený seznam (singly linked list)	10
Třída položky seznamu	11
Třída propojeného seznamu	12
Vytvoření prázdného seznamu	12
Přidávání položek na konec	12
Nastavení současné položky na začátek	13
Získání dat ze současné položky	13
Přepsání dat současné položky	14
Získání další položky	14
Vytvoření iterátoru ze seznamu	15
Hledání položky podle obsahu	15
Odstranění položky podle obsahu dat	16
Zásobník (stack)	17
Třída položky zásobníku	18
Třída zásobníku	18
Test zásobníku jednotlivě	19
Test zásobníku v cyklu	19
Souvislost zásobníku a rekurze	20
Zásobník bez rekurze	20
Rekurze s neviditelným zásobníkem	20
Kompozice tříd	21
Dědičnost tříd	21
Jednoduchá dědičnost	22
Vytvoření instance voláním odvozené třídy	22
Vyvolání zděděné (odvozené) metody	22
Odkaz na základní třídu – super()	23
Překrývání metod (overriding)	23
Více úrovní jednoduché dědičnosti	24
Dvojnásobná dědičnost	25

Vícenásobná dědičnost	27
Metoda volaná se jménem třídy	27
Metoda volaná s funkcí super()	28

Použité zdroje informací

Data Structures and Algorithms Using Python, Rance D. Necaise:

https://doc.lagout.org/science/0_Computer%20Science/2_Algorithms/

[Data%20Structures%20and%20Algorithms%20using%20Python%20%5BNecaise%202010-12-21%5D.pdf](https://doc.lagout.org/science/0_Computer%20Science/2_Algorithms/Data%20Structures%20and%20Algorithms%20using%20Python%20%5BNecaise%202010-12-21%5D.pdf).

Třída (class)

Vlastní objekt můžeme vytvořit definicí třídy (class). Jméno třídy začíná podle dohody velkým písmenem.

```
class Jméno:  
    blok příkazů
```

Ukázka jednoduché třídy

```
class Třida:  
  
    x = 12 # proměnná třídy 1  
  
    def inst_metoda(self, obj): # funkce - instanční metoda 2  
        return obj
```

- 1 Jméno `x` definované uvnitř těla třídy je lokální a nazývá se *proměnná třídy* nebo *třídní proměnná* (class variable).
- 2 Funkce `inst_metoda()` definovaná uvnitř těla třídy, která má první parametr jménem `self`, se nazývá *instanční metoda* (instance method). Symbol `self` je konvenční jméno pro *instanci* své vlastní třídy, tedy třídy (`Třida`), v níž je metoda definována.

```
print(Třida) # <class '__main__.Třida'> 3  
print(Třida.x) # 12 — hodnota proměnné třídy
```

- 3 V tomto okamžiku již existuje *třída* (objekt) jménem `Třida`.

Instance třídy

Voláním třídy se ze třídy vytvoří objekt zvaný *instance*. Volání třídy má stejný tvar jako volání funkce. Instancí lze vytvořit libovolné množství.

```
tr = Třida() # tr je instance třídy 4  
print(tr) # <__main__.Třida object at 0x108abeeb0> 5  
print(tr.x) # 12 6  
print(tr.inst("abc")) # abc 7
```

- 4 *Voláním třídy* `Třida()` vzniká objekt zvaný *instance* třídy `Třida`.
- 5 Jméno `tr` představuje objekt – *instanci* třídy `Třida`.
- 6 Výraz `tr.x` odkazuje na hodnotu proměnné `x` v instanci `tr`. Instance má také přístup k proměnné třídy.
- 7 Výraz `tr.inst("abc")` je volání *instanční metody* `inst_metoda()` s jedním argumentem. Volání vrátí hodnotu zapsaného argumentu. Parametr `self` se ve volání nezapisuje.

Terminologie tříd

Třída obsahuje tzv. *atributy*. Jsou to jednak *datové* atributy – proměnné a jednak *funkční* atributy – metody.

Důležitý je pojem *instance* jako *objektu* vytvořeného ze třídy, přičemž třída je také objekt (ale jiný).

Významy pojmů *třída*, *typ*, *objekt*, *instance* (class, type, object, instance) se během vývoje jazyka Python měnily a jejich současný význam je potřeba si přesně zjistit v oficiální dokumentaci. Jsou si významem podobné a některé se často zaměňují nebo se užívají podle volby. Rozdíly ve významu nejsou k programování důležité; třeba to, že instance třídy není instancí typu, ale je instancí objektu.

```
print(isinstance(tr, type)) # False
print(isinstance(tr, object)) # True
```

Úprava třídy

```
Trida.z = 99 # přidá proměnnou do třídy
print(Trida.z + 100) # 199
del Trida.z # odstraní proměnnou ze třídy

def metoda(self, x, y): # funkce s prvním parametrem self
    return max(x, y)

Trida.metoda = metoda # funkce je přidána do třídy
print(Trida.metoda(None, 1, 2)) # 2 – první libovolný argument se neuplatní
del Trida.metoda # odstranění funkce ze třídy
```

Úprava instance

```
tr.y = 88 # přidá proměnnou do instance
print(tr.y + 100) # 188
del tr.y # odstraní proměnnou z instance

tr.metoda = metoda # funkce je přidána do instance jako metoda
print(tr.metoda(None, 2, 3)) # 3 – první libovolný argument se neuplatní
del tr.metoda # odstranění metody z instance
```

Statické a třídní metody

Kromě instančních metod existují ještě další dva druhy metod. *Statická metoda* (static method) se označuje symbolem `@staticmethod`. *Třídní metoda* nebo *metoda třídy* (class method) se označuje symbolem `@classmethod`.

```
class Trida:

    x = 12 # proměnná třídy

    def __init__(self, par): 1
        self.p = par # instanční proměnná vzniklá z parametru
        self.y = 83 # instanční proměnná 2

    @staticmethod
    def stat(): # statická metoda 3
        return ("statická")

    @classmethod
    def clas(cls): # metoda třídy 4
        return (cls.x, "třídní")
```

- 1 Metoda `__init__()` slouží k *inicializaci* instance třídy a k *vytváření instančních proměnných*. Můžeme jí říkat *inicializátor*. Někdy se nazývá *konstruktor*. Kromě parametru `self` může mít i další parametry. Instanční proměnná `p` je kopií parametru `par`. Tato metoda se vyvolá automaticky při volání třídy.
- 2 Instanční proměnné se používají v instančních metodách a musí mít předponu `self` s tečkou. Instanční proměnná `y` je samostatná s hodnotou 83.
- 3 *Statická metoda* je označena na předchozím řádku symbolem `@staticmethod`. Nezávisí na třídě a chová se stejně jako obyčejná funkce zapsaná mimo třídu; nemá přístup k žádným proměnným (třídním ani instančním). Proto se o ní vedou debaty, k čemu vlastně je.
- 4 *Třídní metoda* nebo *metoda třídy* (class method) je označena na předchozím řádku symbolem `@classmethod`. Závisí na třídě, proto je prvním parametrem symbol `cls`, což je konvenční jméno pro svou vlastní třídu, tedy třídu v níž je metoda definována. Třídní metoda má přístup k třídním proměnným, ale ne k instančním proměnným. Třídní metoda `clas()` vrací dvojici, jejíž první člen je hodnota třídní proměnné `x` označené jako `cls.x`. Třídní proměnné uvnitř třídní metody musí mít předponu `cls` s tečkou.

```
print(Trida.clas() # (12, 'třídní')
print(tr.clas() # (12, 'třídní')

print(Trida.stat() # statická
print(tr.stat() # statická
```

Výrazy `Trida.clas()` i `tr.clas()` jsou voláním *třídní* metody `clas()` bez argumentu. První argument `cls` se ve volání nezapisuje. Třídní metoda `clas()` funguje i v instanci třídy `tr`, protože nepoužívá žádné instanční proměnné.

Televizor a ovladač

Na příkladu symbolického televizoru a ovladače si ukážeme jeden způsob, jak se mohou používat třídy a jejich instance (objekty). Dvě třídy, **Televizor** a **Ovladac** poslouží k vytváření objektů televizorů a jejich ovladačů. Objekty pak spolupracují tak, že ovladač vyšle požadavek a televizor zareaguje. Televizor kontroluje, zda ovladač má stejnou obchodní značku.

```
class Televizor:
    def __init__(self, znacka): # iniciační metoda - konstruktor
        self.znacka = znacka # instanční proměnná z argumentu
        self.zapnut = False # instanční proměnná - stav televizoru
    def cidlo(self, ovladac): # instanční metoda
        if ovladac.znacka != self.znacka:
            print(f"Televizor {self.znacka} "
                  f"neposlouchá ovladač {ovladac.znacka}!")
            return
        if self.zapnut: # odkaz na proměnnou instance své třídy (Televizor)
            print(" jsem zapnutý, vypínám se")
            self.zapnut = False
        else:
            print("zapínám se")
            self.zapnut = True
```

```
class Ovladac:
    def __init__(self, znacka):
        self.znacka = znacka # instanční proměnná z argumentu
    def zapni_televizor(self, tel): # stisk tlačítka ⏻
        # volá metodu instance třídy Televizor se sebou samým jako argumentem
        tel.cidlo(self)
```

Test televizoru a ovladače

```
# vytvořím instanci třídy Televizor
tel = Televizor(znacka="LG")

# vytvořím se instanci třídy Ovladac
ovl = Ovladac(znacka="LG")

# volám instanční metodu ovladače; argument je instance třídy Televizor
ovl.zapni_televizor(tel) # zapínám se
ovl.zapni_televizor(tel) # jsem zapnutý, vypínám se

# vytvořím jinou instanci třídy Ovladac
ovl = Ovladac(znacka="Samsung")
ovl.zapni_televizor(tel) # Televizor LG neposlouchá ovladač Samsung!
```

Každá instance třídy **Televizor** i **Ovladac** má svou vlastní, stálou značku (instancční proměnná `znacka`), zatímco stav instance (instancční proměnná `zapnut`) se mění při každém volání čidla.

Vstupní parametry třídy a instanční proměnné

Aby bylo možné pro vytvoření instance ve volání třídy zadat *vstupní parametry*, zařazuje se do těla třídy speciální metoda s názvem `__init__()`, která přebírá argumenty volání a vytváří z nich instanční proměnné. Umožňuje také vytvářet samostatné instanční proměnné nezávislé na parametrech.

```
class Televizor:
    def __init__(self, znacka): # iniciační metoda - konstruktor
        self.znacka = znacka # instanční proměnná s hodnotou z argumentu
        self.zapnut = False # instanční proměnná - stav televizoru
```

Instanční proměnnou je zvykem pojmenovat stejně jako parametr, i když to není povinné.

Slovo **self** s tečkou je nutné vždy předřadit před jméno instanční proměnné vlastní třídy.

```
class Televizor:
    ...
    def cidlo(self, ovladac):
        if ovladac.znacka != self.znacka:
            ...
```

`self.znacka` je odkaz na instanční proměnnou *vlastní* třídy Televizor, zatímco `ovladac.znacka` je odkaz na instanční proměnnou z *cizí* třídy Ovladac. Objekt ovladac je převzat zvenčí, z argumentu volání metody; před slovo ovladac tedy *nepatří* slovo self.

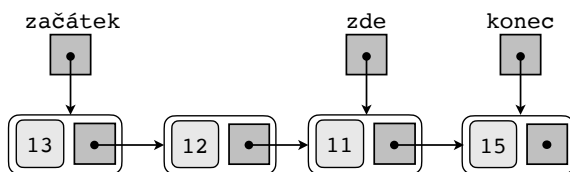
Datové struktury

V kurzech programování jsou jako datové struktury běžně probírány *propojené seznamy* a *zásobníky*. Ukážeme si třídy a metody, jimiž se dají vytvářet a zpracovávat. K vytváření a zpracování takových struktur bychom také mohli použít seznam (list), ale neviděli bychom jejich vnitřní uspořádání.

Jednostranně propojený seznam (singly linked list)

Na příkladu jednostranně propojeného seznamu ukážeme, jak se realizují a používají ukazatele (odkazy, pointers). Jde o seznam objektů – datových položek, které na sebe vzájemně odkazují, a to tak, že jeden odkazuje (ukazuje) na další, atd. To znamená, že když známe odkaz na jeden, můžeme získat odkaz na další, atd., a tak se postupně dostat k libovolné položce. Ukazatele tedy ukazují jen jedním směrem: na další položku.

Na obrázku je znázorněn propojený seznam položek číselnými daty (13, 12, 11, 15) a s vyznačenými ukazateli na začátek, na konec, a na současně zpracovávanou pozici (zde). Tečka s šipkou značí ukazatel na položku, tečka bez šipky u poslední položky značí prázdný ukazatel (None).



Třída položky seznamu

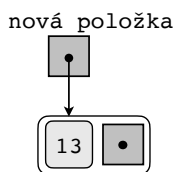
K vytváření položek si zavedeme třídu `Polozka` reprezentující položku seznamu.

```
class Polozka:
    def __init__(self, data):
        self.data = data
        self.dalsi = None
```

Proměnná `self.data` bude obsahovat hodnotu převzatou z argumentu. Proměnná `self.dalsi` bude sloužit jako *ukazatel na další položku* (zpočátku prázdný). Pro jednoduchost budeme předpokládat, že data budou jen celá čísla, ačkoliv to mohou být libovolné objekty.

Novou položku (instanci třídy `Polozka`) s číslem 13 a prázdným odkazem na další položku můžeme vytvořit následujícím příkazem.

```
nova_polozka = Polozka(13)
print(nova_polozka.data)  # 13
```



Proměnná `nova_polozka` ukazuje (tečka s šipkou) na nově vytvořenou položku. Ukazatel na další položku je prázdný (tečka bez šipky). Je to instanční proměnná `self.dalsi` s hodnotou `None`.

Třída propojeného seznamu

Abychom mohli vytvářet a zpracovávat propojené seznamy, zavedeme si třídu, která je reprezentuje. Nazveme ji `LinkedList`. Ze začátku bude obsahovat jen metodu `__init__`, další metody přidáme později.

```
class LinkedList:
    def __init__(self):
        self.zacatek = None # začátek seznamu
        self.konec = None # konec seznamu
        self.zde = None # současná pozice
```

Vytvoření prázdného seznamu

```
lst = LinkedList()

print(lst.zacatek) # None
print(lst.konec) # None
print(lst.zde) # None
```

Přidávání položek na konec

Metoda **`append()`** přidá novou položku na *konec*. Dosavadní položky, pokud existují, nechává na místě, proměnná *konec* bude ukazovat na poslední položku.

```
def append(self, data):
    nova_polozka = Polozka(data) # instance nové položky
    if self.konec is not None: # neprázdný seznam? 1
        self.konec.dalsi = nova_polozka # ukazatel na další
    else: # prázdný seznam? 2
        self.zacatek = nova_polozka # přidám novou položku na začátek
    self.konec = nova_polozka # ukazatel na konec 3
    self.zde = nova_polozka # ukazatel na současnou pozici 4
    return self.konec 5
```

1 Je-li seznam *neprázdný*, dosadí v koncové položce do proměnné *dalsi* novou položku jakožto ukazatel na další položku.

2 Je-li seznam *prázdný*, dosadí do proměnné *zacatek* novou (první) položku.

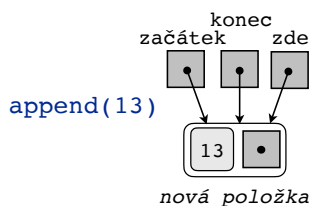
3 Dosavadní položku *konec* nahradí ukazatel na novou položku. Nová položka teď představuje konec seznamu.

4 Do položky *zde* na současnou pozici dosadí také ukazatel na novou položku. Obě položky, *zde* i *konec*, budou mít prázdný ukazatel na další položku (*self.dalsi = None*).

5 Metoda vrací ukazatel na konec seznamu (ale také na současnou pozici).

Do seznamu zařazujeme postupně nové položky metodou `append()`. Nejprve první položku do prázdného seznamu, přičemž všechny tři ukazatele (začátek, konec, zde) ukazují na novou položku.

```
lst.append(13)
```

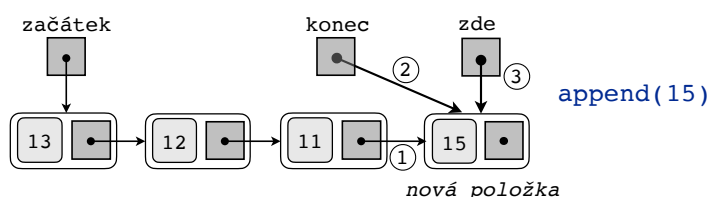


Pak se ostatní položky zařazují již do neprázdného seznamu a ukazatele se mění.

```
lst.append(12)
```

```
lst.append(11)
```

```
lst.append(15)
```



Tímto postupem jsme vytvořili propojený seznam `lst`, který můžeme dále zpracovávat, když třídu `LinkedList` doplníme o další metody. Budeme potřebovat změnit ukazatele na začátek, na konec nebo na současnou pozici, aniž bychom při tom používali instanční proměnné. Dále chceme vyhledat položku podle obsahu, přecházet další položku v pořadí a odstranit určenou položku.

Nastavení současné položky na začátek

Metoda `set_start()` nastaví ukazatel na současnou položku (jestliže existuje) na začátek seznamu a tento ukazatel vrátí. Jinak vrátí `None`.

```
def set_start(self):
    self.zde = self.zacatek
    return self.zde
```

Získání dat ze současné položky

Metoda `get_data()` vrátí data ze současné položky (na niž ukazuje proměnná `zde`) nebo `None`.

```
def get_data(self):
    if self.zde is not None:
        return self.zde.data
    else:
        return None
```

Přepsání dat současné položky

Metoda `set_data()` přepíše data současné položky (jestliže existuje) hodnotou z argumentu a vrátí nová data. Jinak vrátí `None`.

```
def set_data(self, data):
    if self.zde is not None:
        self.zde.data = data
        return self.zde
    else:
        return None
```

Získání další položky

Metoda `__next__()` přečte ze současné položky (na niž ukazuje proměnná `zde`) ukazatel na *následující položku*, dosadí jej zpět do proměnné `zde` (pro případné příští použití metody `__next__()`) a ukazatel vrátí. Metoda `__next__()` spolu s metodou `__iter__()` slouží k tomu, aby se třída mohla použít jako *iterátor*.

```
def __next__(self):
    # u prázdného seznamu vydá výjimku MemoryError
    if self.zde is None:
        raise MemoryError ("Seznam je prázdný.")
    # na konci seznamu vydá výjimku StopIteration
    elif self.zde.dalsi is None:
        raise StopIteration
    else:
        self.zde = self.zde.dalsi # další položka bude současná
        return self.zde # vrátí následující položku
```

Následující program nastaví začátek seznamu a přečte první položku, a pak v cyklu vytiskne data všech položek.

```
p = lst.set_start()
while p:
    try:
        print(f"{lst.get_data()}", end=" ") # 13 12 11 15
        p = lst.__next__() # získá se další položka
    except StopIteration:
        break
```

Vytvoření iterátoru ze seznamu

Metoda `__iter__()` zajišťuje umělou "nultou" položku tak, aby *současná položka* ukazovala na začátek seznamu. Vrací instanci své třídy (objekt `self`). Metoda `__iter__()` spolu s metodou `__next__()` slouží k tomu, aby se třída mohla použít jako *iterátor*.

```
def __iter__(self):
    nulta_polozka = Polozka(None)
    nulta_polozka.dalsi = self.zacatek
    self.zde = nulta_polozka
    return self
```

Následující úryvek programu ukazuje, jak se dá propojený seznam použít jako *iterátor*. Iterátor vznikne z instance třídy použitím vestavěné funkce `iter()`, která interně volá metodu `__iter__()`. V cyklu `for-in` se vytisknou data všech položek seznamu.

```
list_iterator = iter(lst) # vytvoření iterátoru
for _ in list_iterator:
    print(f"{lst.get_data()}", end=" ") # 13 12 11 15
```

Hledání položky podle obsahu

Metoda `find()` hledá *od začátku* položku s daným obsahem dat. Když ji najde, nastaví ukazatel na nalezenou položku do proměnné `zde` a vrátí ukazatel. Když ji nenajde, do proměnné `zde` dosadí `None` a vyvolá výjimku `Exception`.

```
def find(self, data):
    self.zde = self.zacatek
    while self.zde is not None and self.zde.data != data:
        self.zde = self.zde.dalsi
    if self.zde is None:
        raise Exception (f"Položka {data} se nenalezla.")
    return self.zde
```

```
lst.find(11)
print("nalezená data:", lst.get_data()) # nalezená data: 11
```

```
lst.find(12345)
lst.get_data() # Exception: Položka 12345 se nenalezla.
```

Odstranění položky podle obsahu dat

Metoda **remove()** odstraní položku, která obsahuje zadaná data. Hledá ji od začátku. Najde-li ji, ze této položky ukazatel na následující položku a umístí jej do předchozí položky, čímž se ta stane současnou a nalezená položka se ztratí (nemíří na ni žádný ukazatel). Vráti data odstraněné položky.

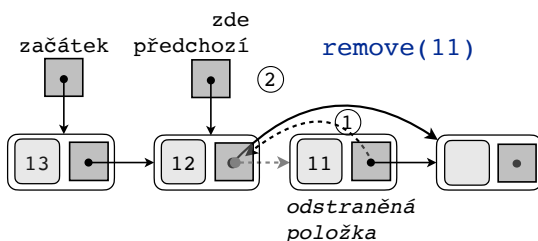
```
def remove(self, data):
    pred_polozka = None # prázdný ukazatel na počáteční předchozí položku
    self.zde = self.zacatek
    # hledám odstraňovanou položku

    while self.zde is not None and self.zde.data != data: 1
        pred_polozka = self.zde # pamatuji si předchozí položku 2
        self.zde = self.zde.dalsi

    if self.zde is None: 3
        raise StopIteration(f"Odstraňovaná položka {data} se nenalezla.")
    odstranena = self.zde # nalezená položka
    pred_polozka.dalsi = self.zde.dalsi 4
    self.zde = pred_polozka # předchozí položka se stane současnou 5
    return odstranena.data # vrací data odstraněné položky
```

- 1 Metoda nejprve hledá položku s danými daty.
- 2 Při hledání si pamatuje ukazatel na předchozí položku.
- 3 Nenajde-li hledanou položku, vyhodí výjimku `StopIteration`.
- 4 Najde-li hledanou položku, zjistí z ní ukazatel na další položku a zapíše ho do předchozí položky (i kdyby byl `None`, například u poslední položky).
- 5 Ukazatel na současnou položku `zde` se stane ukazatelem na předchozí položku.

```
odstranena = lst.remove(11)
print("odstranena:", odstranena) # odstranena: 11
```

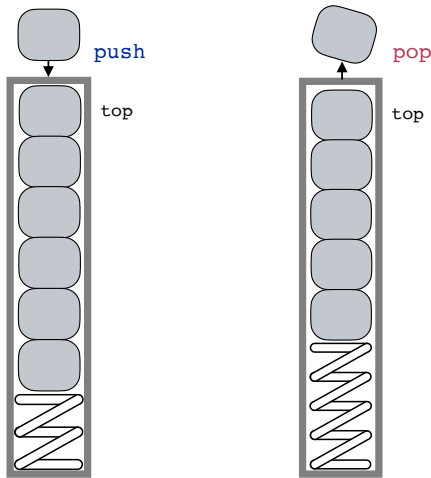


Výsledek ověříme hledáním odstraněné položky metodou `find()` nebo opakováním stejného příkazu.

```
try:
    print("find:", lst.find(11))
except StopIteration as err:
    print(err) # Položka 11 se nenalezla
```


Zásobník (stack)

Zásobník je zařízení pro ukládání předmětů ve vydávacím automatu nebo nábojů v pistoli. Způsob ukládání a odběru se označuje LIFO (Last In, First Out) neboli: poslední, který byl vtlačen dovnitř (*push*), bude první odebrán (*pop*). Začátek (vršek) zásobníku se obvykle nazývá *top*.



V programování se zásobník používá tehdy, kdy je zapotřebí dočasně uchovat mezivýsledky zpracování, zejména u vnořených datových struktur, například při výpočtu aritmetického výrazu se závorkami. Zásobník (stack) v programu je datová struktura, která umožňuje ukládat datové položky jednu po druhé a vybírat je v obráceném pořadí. K tomu jsou zapotřebí tři operace:

- zápis položky do zásobníku: `push ( )` – vtlač
- odebrání položky ze zásobníku: `pop ( )` – vyraž a vrať položku
- dotaz na obsah zásobníku: `empty ( )` – prázdný

Někdy se přidávají další operace:

- zjištění první položky (top): `peek ( )` – vršek
- zjištění délky zásobníku: `len ( )` – délka

Také se někdy omezuje délka zásobníku mezní hodnotou.

Objekt zásobníku a objekty položek se vytvoří pomocí tříd, podobně jako propojený seznam. Operace spočívají v manipulaci s ukazateli na *první položku* (začátek) a *další položku*.

Třída položky zásobníku

Je stejná jako položka propojeného seznamu (výše).

```
class Polozka:
    def __init__(self, data):
        self.data = data
        self.dalsi = None # ukazatel na další položku
```

Třída zásobníku

Ve srovnání s propojeným seznamem je krátká.

```
class Zasobnik:

    # Vytvoří nový zásobník s prázdným začátkem.
    def __init__(self):
        self.top = None # ukazatel na první položku

    # Vrátí True, je-li zásobník prázdný, jinak vrátí False.
    def empty(self):
        return self.top is None

    # Odstraní položku ze začátku zásobníku a vrátí její data.
    def pop(self):
        if self.empty():
            raise MemoryError(f"Zásobník {self} je prázdný!")
        stara_top = self.top # schová odebíranou první položku
        self.top = stara_top.dalsi # nahradí odebíranou položku další položkou
        return stara_top.data # vrátí data odebrané položky

    # Vtlačí (přidá) novou položku na začátek (top) zásobníku.
    def push(self, data) :
        nova_polozka = Polozka(data) # vytvoří novou položku
        nova_polozka.dalsi = self.top # doplní do ní ukazatel na další položku
        self.top = nova_polozka # nová položka se stane začáteční
```

Test zásobníku jednotlivě

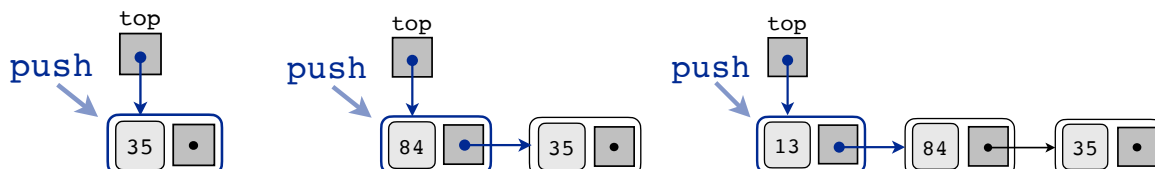
```
zasobnik = Zasobnik() # vytvoření instance prázdného zásobníku
```



```
zasobnik.push(35) # první položka do prázdného zásobníku
```

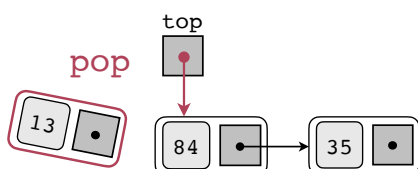
```
zasobnik.push(84) # druhá položka zatlačí první
```

```
zasobnik.push(13) # třetí položka zatlačí druhou a první
```



```
# první položka ven, ostatní se posunou dopředu
```

```
print("pop", zasobnik.pop()) # pop: 13
```



Test zásobníku v cyklu

```
i = 2
while i >= 0: # naplnění zásobníku čísly 2, 1, 0
    zasobnik.push(i)
    i -= 1
```

```
while True: # vyprázdnění zásobníku v cyklu
    try:
        zasobnik.pop()
    except MemoryError as err:
        print("Chyba!", err)
        break
```

```
2
1
0
0
1
2
```

```
Chyba! Zásobník <__main__.Zasobnik object at 0x10c2adfd0> je prázdný!
```

Souvislost zásobníku a rekurze

Zásobník bez rekurze

```
def push_pop(n):
    for i in range(n+1):
        print("tam:", n-i)
        zasobnik.push(n-i) # n-0, n-1, ..., n-n do zásobníku
    while True:
        try:
            i = zasobnik.pop()
            print("ven:", i) # 0, 1, ..., n ze zásobníku
        except:
            break
```

```
push_pop(2)
```

```
tam: 2
tam: 1
tam: 0
ven: 0
ven: 1
ven: 2
```

Rekurze s neviditelným zásobníkem

Také při rekurzi se hodnoty nejprve ukládají do interního zásobníku a při výstupu z rekurze se z něj vybírají v obráceném pořadí.

```
def tam_ven(n) :
    if n >= 0 :
        print("tam:", n) # tiskne při vstupech do funkce
        tam_ven(n - 1)
        print("ven:", n) # tiskne při návratech z funkce (výstup z rekurze)
```

```
tam_ven(2)
```

```
tam: 2
tam: 1
tam: 0
ven: 0
ven: 1
ven: 2
```

Kompozice tříd

Třídy se mohou navzájem používat. Kupříkladu třída `Zasobnik` používá třídu `Polozka` ve formě *instance*, kterou si vytvoří voláním třídy `Polozka(data)`:

```
def push(self, data) :  
    nova_polozka = Polozka(data) # nová položka  
    ...
```

Nebo třída `Televizor` používá třídu `Ovladac` ve formě instance `ovladac` přijímané jako *argument volání* metody `cidlo()`:

```
class Televizor:  
    ...  
    def cidlo(self, ovladac): # instanční metoda  
        if ovladac.znacka != self.znacka:  
            ...
```

Obráceně, třída `Ovladac` zase používá třídu `Televizor` ve formě instance `tel` přijímané jako *argument volání* metody `zapni_televizor()`:

```
class Ovladac:  
    ...  
    def zapni_televizor(self, tel):  
        tel.cidlo(self) # volá metodu instance se sebou jako argumentem
```

Takovému vztahu tříd se říká *kompozice* (composition).

Dědičnost tříd

Kromě jednostranného nebo vzájemného použití při kompozici mohou být třídy ve vztahu *základní* a *odvozená*. Zapisuje se to u definice odvozené třídy uvedením základní třídy v závorce:

```
class Odvozena(Zakladni):
```

Takovému vztahu se říká *dědičnost* (inheritance), přičemž se používá se různé názvosloví.

- *Základní* třída (base class) se často nazývá *nadtřída* (superclass), *předek* (predecessor), *rodičovská třída* (parent class).
- *Odvozená* třída (derived class) se nazývá *podtřída* (subclass), *potomek* (ancestor), *dětská třída* (child class) a *dědí* od základní třídy data i metody.

Odvozená třída může data a metody základní třídy nejen používat, ale i modifikovat.

Následující příklad ukazuje, jak odvozená třída může používat metodu základní třídy a modifikovat instanční data základní třídy.

Jednoduchá dědičnost

```
class Zakladni:
    def __init__(self): 3
        self.data = "data základní třídy"
    def metoda(self):
        print(f"základní metoda tiskne: {self.data}") 4
```

```
class Odvozena(Zakladni):
    def __init__(self): 1
        self.data_o = " + data odvozené třídy" 2
        Zakladni.__init__(self) # vyvolám inicializaci třídy Zakladni 3
        self.data += self.data_o 4
```



Vytvoření instance voláním odvozené třídy

```
o = Odvozena()
```

- Při inicializaci odvozené třídy 1 se nejdříve vytvoří její instanční data 2.
- Volání metody `Zakladni.__init__(self)` provede inicializaci základní třídy, tedy vytvoření její instance 3.
- Tím se stalo, že výraz `self.data` je platný i v odvozené třídě a představuje *data zděděná* ze základní třídy.
- Nakonec se data základní třídy modifikují přidáním dat z odvozené třídy 4.

Vyvolání zděděné (odvozené) metody

```
o.metoda()
základní metoda tiskne: data základní třídy + data odvozené třídy
```

Protože odvozená třída dědí metody i data, můžeme z ní tedy volat metodu `metoda()`, přestože ji nemá v sobě definovanou.

Odkaz na základní třídu – `super()`

Inicializátor základní třídy `Zakladni` jsme volali přímo, s uvedením instance `self` v argumentu metody:

```
Zakladni.__init__(self).
```

Kdybychom ale při údržbě programů potřebovali třídu `Zakladni` přejmenovat nebo ji nahradit jinou třídou, museli bychom v programu jméno třídy přepsat. Tomu se lze vyhnout použitím funkce `super()`, která *nahrazuje jméno základní třídy* při volání jejího inicializátoru a vypouští jméno `self` z argumentu:

```
super().__init__()
```

Překrývání metod (overriding)

Jestliže se metoda v odvozené třídě jmenuje stejně jako v základní třídě (zde `metoda()`), mluví se o *překrývání metod* (overriding). Vyvolá-li se stejnojmenná metoda z instance odvozené třídy, provede se metoda definovaná v odvozené třídě. Má tedy *přednost* před metodou ze základní třídy.

Pro stručnost vynecháme data obou tříd, ale použijeme místo nich metodu s parametrem:

```
class Zakladni:
    def metoda(self, parametr):
        print(f"základní třída přijímá parametr: '{parametr}'")

class Odvozena(Zakladni):
    def metoda(self, parametr): # překrývá metodu v třídě Základní
        print(f"odvozená třída přijímá parametr: '{parametr}'")
        super().metoda("z odvozené třídy") # volá základní metodu
```

```
o = Odvozena()
```

```
o.metoda("zvenku")
```

```
odvozená třída přijímá parametr: 'zvenku'
```

```
základní třída přijímá parametr: 'z odvozené třídy'
```

Nejdříve se vyvolá metoda *odvozené* třídy s argumentem `"zvenku"` a otiskuje jej. V této metodě se dále pomocí funkce `super()` volá stejnojmenná metoda *základní* třídy s jiným argumentem a ta ho také otiskne.

Více úrovní jednoduché dědičnosti

Třídy mohou vzájemně dědit ve více úrovních. Mějme tři třídy A, B, C, které tvoří hierarchii

A nejvyšší předek

B jeho potomek a předek dalšího potomka

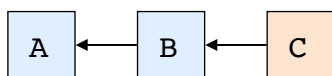
C poslední potomek

```
class A:
    def metoda(self):
        print(f"jsem třída A, předek je '{A.__base__.__name__}'"
              f" a potomek všech je {self.__class__.__name__}")
```

```
class B(A):
    def metoda(self):
        print(f"jsem třída B, předek je '{B.__base__.__name__}'")
        super().metoda()
```

```
class C(B):
    def metoda(self):
        print(f"jsem třída C, předek je '{C.__base__.__name__}'")
        super().metoda()
```

Symbol `A.__base__` představuje základní třídu třídy A. Symbol `self.__class__` představuje třídu objektu `self`. Symbol `__name__` představuje jméno třídy.



Jako příklad vytvoříme instanci `c` posledního potomka C (poslední odvozené třídy) a z ní vyvoláme metodu, která má stejné jméno `metoda` i u předků, proto je překrývá. Každá z nich volá metodu předka prostřednictvím funkce `super()`.

```
c = C() # instance posledního potomka
c.metoda()
jsem třída C, předek je 'B'
jsem třída B, předek je 'A'
jsem třída A, předek je 'object' a potomek všech je C
```

Všimněme si, že třídy A, B nebyly inicializovány a nemají tedy svoje instance. To znamená, že na konci série volání v třídě A se jako potomek prosadí třída C. Je to tím, že proměnná `self` vznikla jako odkaz na instanci `c` třídy C při jejím vytvoření, takže jméno `self` stále ukazuje na C.

Třída A nejvyšší úrovní je podtřídou třídy `object`. Definici třídy A můžeme tedy napsat také se jménem `object` v závorce:

```
class A(object):
    ...
```


Dvojnásobná dědičnost

Třída může dědit zároveň ze dvou jiných tříd. Například zápis

`T (T1, T2)`

znamená, že třída `T` dědí data a metody nadtříd `T1` a `T2`.

V následujícím příkladu máme třídu `A`, která obsahuje sčítací metodu `plus()` a třídu `B`, která má násobící metodu `krat()`. Třída `C` dědí obě třídy `A` a `B` a může tedy používat obě metody.

```
class A:
    def __init__(self):
        print(f"inicializace A")

    def plus(self, a, b):
        return a + b

class B:
    def __init__(self):
        print(f"inicializace B")

    def krat(self, a, b):
        return a * b

class C (A, B):
    def __init__(self):
        print(f"začátek inicializace C")
        A.__init__(self)
        B.__init__(self)
        print(f"konec inicializace C")
```

```
c = C() # instance třídy C
začátek inicializace C
inicializace A
inicializace B
konec inicializace C

print(f"{c.plus(1, 2) + c.krat(1, 2) = }")
c.plus(1, 2) + c.krat(1, 2) = 5
```

V třídě `C` se volá inicializátor obou nadtříd s uvedením jména třídy. To znamená, že třídy `A`, `B` dostanou každá svou vlastní proměnnou `self`. Parametry `a`, `b` jsou uvnitř obou metod použity bez instance své třídy (není u nich předpona `self`).

Druhý příklad je stejný, ale v třídě C se inicializátor nadtříd se volá pomocí funkce `super()`. První volání `super()` bez argumentu vyvolá inicializátor první nadtříd A. K vyvolání inicializátoru nadtříd B pomocí `super()` je zapotřebí nejprve vytvořit její instanci, třeba `be = B()` a tu teprve použít jako druhý argument ve volání funkce `super(B, be)`. První argument musí být třída B. Druhý argument ve skutečnosti může být podtřídou třídy B, takovou ale nemáme. Pravidla pro funkci `super()` najdeme v dokumentaci <https://docs.python.org/3/library/functions.html#super>. Parametry metod se tentokrát na ukázkou převádějí na instanční proměnné (s předponou `self`).

```
class A:
    def __init__(self):
        self.a = 0
        self.b = 0
        print(f"inicializace A")

    def plus(self, a, b):
        self.a = a
        self.b = b
        return self.a + self.b
```

```
class B:
    def __init__(self):
        self.a = 0
        self.b = 0
        print(f"inicializace B")

    def krat(self, a, b):
        self.a = a
        self.b = b
        return self.a * self.b
```

```
class C(A, B):
    def __init__(self):
        print(f"začátek inicializace C")
        super().__init__()
        be = B()
        super(B, be).__init__()
        print(f"konec inicializace C")
```

```
c = C()
začátek inicializace C
inicializace A
inicializace B
konec inicializace C

print(f"{c.plus(1, 2) + c.krat(1, 2) = }")
c.plus(1, 2) + c.krat(1, 2) = 5
```

Vícenásobná dědičnost

Zápis několika tříd T1, T2, ... oddělených čárkou v závorce v definici odvozené třídy T určuje vícenásobnou dědičnost.

Metoda volaná se jménem třídy

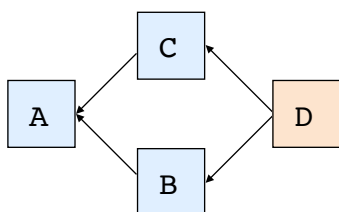
Následující příklad ukazuje, v jakém pořadí se provádí stejnojmenná metoda volaná se jménem třídy. Symbol `__base__` označuje *první* třídu zapsanou v závorce definice odvozené třídy. Tentokrát není ve třídách definován ani volán inicializátor `__init__()`.

```
class A:
    def metoda(self):
        print(f"jsem metoda třídy A "
              f"a tisknu název potomka {self.__class__.__name__}")
```

```
class B(A):
    def metoda(self):
        print(f"jsem metoda třídy B "
              f"a překrývám metodu předka {B.__base__.__name__}")
        A.metoda(self) # volám metodu předka
```

```
class C(A):
    def metoda(self):
        print(f"jsem metoda třídy C "
              f"a překrývám metodu předka {C.__base__.__name__}")
        A.metoda(self) # volám metodu předka
```

```
class D(B, C):
    def metoda(self):
        print(f"jsem metoda třídy D "
              f"a překrývám metody předka {D.__base__.__name__}")
        B.metoda(self) # volám metodu předka
        C.metoda(self) # volám metodu předka
```



```
d = D()
d.metoda()
jsem metoda třídy D a překrývám metodu předka B
jsem metoda třídy B a překrývám metodu předka A
jsem metoda třídy A a tisknu název potomka D
jsem metoda třídy C a překrývám metodu předka A
jsem metoda třídy A a tisknu název potomka D
```

Zde se metoda třídy A volá *dvakrát* (z třídy B i C) a pořadí je D, B, A, C, A. Kdybychom v třídě D zaměnili pořadí volání metody na

```
C.metoda(self) # volám metodu předka
B.metoda(self) # volám metodu předka
```

dostali bychom pořadí D, C, A, B, A.

Metoda volaná s funkcí `super()`

Při použití funkce `super()` se stejnojmenná metoda volá v přesně stanoveném pořadí a v každé třídě jen *jednou*. Python stanoví pořadí podle pravidla nazvaného *Multiple inheritance Resolution Order*. (pořadí v řešení násobné dědičnosti) zkráceně *mro*.

```
class A:
    def metoda(self):
        print(f"jsem metoda třídy A "
              f"a tisknu název potomka {self.__class__.__name__}")

class B(A):
    def metoda(self):
        print(f"jsem metoda třídy B "
              f"a překrývám metodu předka {B.__base__.__name__}")
        super().metoda() # volám metodu předka

class C(A):
    def metoda(self):
        print(f"jsem metoda třídy C "
              f"a překrývám metodu předka {C.__base__.__name__}")
        super().metoda() # volám metodu předka

class D(B, C):
    def metoda(self):
        print(f"jsem metoda třídy D "
              f"a překrývám metodu předka {D.__base__.__name__}")
        super().metoda() # volám metodu předka
```

Pravidlo *mro* pro funkci `super()` u násobné dědičnosti říká, že metoda se nejprve vybírá ze tříd v pořadí, v jakém jsou zapsány v definici, a pak teprve z jejich nadtříd. Proto se provede nedřívě metoda z třídy D, pak z třídy B, pak z třídy C, a pak teprve z třídy A. Z třídy B se metoda předka (nadtřídy A) nevolá, a proto se A neopakuje.

```
d = D()
d.metoda()
jsem metoda třídy D a překrývám metodu předka B
jsem metoda třídy B a překrývám metodu předka A
jsem metoda třídy C a překrývám metodu předka A
jsem metoda třídy A a tisknu název potomka D
```

Metoda `mro()` zobrazí pořadí tříd, v jakém jim předává řízení funkce `super()`. V našem příkladu je to pořadí D, B, C, A, object.

```
print(D.mro())
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

Poznámka: Vícenásobná dědičnost je složitá a nebezpečná. Umožňuje totiž nechtěné vícenásobné nebo dokonce rekurzivní volání inicializačních i instančních metod. Proto je nutné používat pravidla kooperace tříd, například podle článku <https://rhettinger.wordpress.com/2011/05/26/super-considered-super/>. Nejlepší je vyhnout se používání vícenásobné dědičnosti vůbec.